LEAN

Origen e Integración con el Movimiento Ágil

- **Alineación Lean-Agile**: Lean y el Manifiesto Ágil no se contraponen, sino que son complementarios. De hecho, frameworks ágiles (Scrum) han asimilado e incorporado activamente principios Lean. Hoy en día se abordan como enfoques integrados **Lean-Agile**.

Los 7 Principios de Lean Software Development y la Eliminación de Desperdicios

1. Eliminar Desperdicios

Es el pilar fundamental. Se define como desperdicio (*waste*) a **todo aquello que consume tiempo, esfuerzo o recursos sin agregar un valor real percibido por el cliente**.

En manufactura tangible, el foco principal era eliminar el exceso de stock e inventario en depósitos. En el desarrollo de software se traducen de la siguiente manera:

Desperdicio en Manufactura	**Desperdicio en Desarrollo de Software**	**Explicación Conceptual y Manifestación**
Sobreproducción	**Características Extra / Requerimientos no usados (Gold Plating)**	Crear código o funcionalidades que nadie solicitó o que nunca se utilizan *Principio de Pareto en Software*: El **80% del valor** de un producto se concentra en el **20% de sus características**
Inventario	**Trabajo Parcialmente Terminado (Partially Done Work)**	Código o especificaciones a medio hacer. Calcular el "porcentaje de avance" en tareas intelectuales es subjetivo e ineficiente, por ello, la gestión empírica exige una medición binaria (Terminado / No Terminado).
Proceso Extra	**Pasos Extra / Burocracia**	Tareas, documentos o especificaciones excesivas al inicio del proyecto. En software, las especificaciones extensas "se ponen viejas" porque las

		necesidades del cliente y el entorno cambian antes de su implementación.
Transporte	**Transferencia de Conocimiento (Handoffs)**	Pérdida de contexto e información relevante cada vez que un trabajo pasa de un rol o equipo a otro.
Movimiento	**Cambio de Tarea / Multitasking / Cambio de Contexto**	Trabajar simultáneamente en múltiples tareas. En el trabajo intelectual, la multitarea es altamente improductiva debido al tiempo de re-configuración mental (*seteo*) necesario para pasar de un problema a otro.
Esperas	**Esperas y Bloqueos (Waiting)**	Tiempos muertos esperando que terceros, proveedores o compañeros entreguen información, APIs o componentes bloqueantes.
Defectos	**Defectos y Re-trabajo**	Errores en el código. En la industria del software el re-trabajo puede representar hasta el **50% del costo total** del desarrollo.
Talento no Utilizado	**Talento no Utilizado**	No aprovechar la experiencia, creatividad y capacidad técnica de los miembros del equipo.

2. Amplificar el Aprendizaje (*Amplify Learning*)

El desarrollo de software es un proceso empírico de construcción de conocimiento. Se promueve transformar el conocimiento implícito en explícito a través de la colaboración, la retroalimentación continua y el intercambio dentro del equipo.

3. Construir Calidad Integrada / Embeber la Integridad Conceptual (*Build Quality In*)

La calidad no se puede "agregar al final" ejecutando más pruebas o testing antes de liberar el producto. **La calidad es una decisión** que debe integrarse de manera continua en cada línea de código y artefacto a lo largo de todo el ciclo de vida.

4. Diferir Compromisos / Tomar Decisiones Informadas (*Defer Commitment*)

Consiste en posponer las decisiones irreversibles o de arquitectura hasta el **último momento responsable** (*last responsible moment*), es decir, hasta el punto óptimo en que se cuenta con la mayor cantidad de información concreta.

5. Empoderar al Equipo (*Empower the Team*)

Fomentar equipos motivados, capacitados y auto-organizados. Se descentraliza la toma de decisiones: el equipo ejecutor es quien decide cómo llevar a cabo el trabajo, en lugar de someterse a un micro-management jerárquico.

6. Ver el Todo / Optimizar el Conjunto (*Optimize the Whole*)

Busca no enfocarse únicamente en líneas de código o pantallas aisladas, sino comprender el valor completo que el sistema aporta al negocio y al cliente.

7. Entregar lo Antes Posible / Realimentación Rápida (*Deliver Fast*)

Realizar entregas rápidas y frecuentes en pequeñas porciones para obtener retroalimentación real del cliente, validar hipótesis y reducir la incertidumbre del mercado.

KANBAN

Kanban **NO es una metodología de desarrollo de software**, ni un proceso prescriptivo de construcción de código. Es un **Framework para la Mejora Continua de Procesos**. Kanban asume que el equipo ya sabe construir software y se aplica sobre el proceso existente para evolucionarlo.

Fundamento

El funcionamiento de Kanban se apoya en la **Teoría de Administración de Colas (Queueing Theory)** y la gestión activa del flujo.

Se mapea el proceso real en un tablero con columnas.

- **Tarjetas**: Representan las unidades de trabajo (historias de usuario, defectos, tareas, spikes).
- **Tipos de Columnas**:
    - *Columnas de Trabajo (Doing)*: Muestran trabajo activo en ejecución
    - *Colas (Done / Ready / Buffer)*: Indican etapas en las que el trabajo se encuentra listo para ser arrastrado a la siguiente fase (ej. *Listo para Dev, Listo para Test*).

Limitar el Trabajo en Progreso (WIP - *Work In Progress*)

Es la regla técnica más estricta del método. Establece el número máximo de tarjetas que una columna o etapa puede albergar simultáneamente.

- **Propósito**: Evitar la saturación del equipo y cuellos de botella, obligar a los miembros del equipo a colaborar para resolver bloqueos antes de iniciar nuevas tareas.
- **Impacto del límite WIP en la Cola Inicial (Backlog)**: Si la columna inicial de Backlog no tiene límite WIP, las ideas se acumulan indefinidamente y el **Lead Time** total del cliente aumenta de forma drástica.

Sistema de Arrastre (*Pull System vs. Push System*)

En lugar de "empujar" (*push*) tareas hacia los desarrolladores, los trabajadores "arrastran" (*pull*) nuevos ítems únicamente cuando se libera capacidad y el límite WIP de su columna lo permite.

Políticas Explícitas

Los criterios para mover una tarjeta entre columnas deben ser públicos y explícitos para todo el equipo (ej. *Definition of Done*, criterios de aceptación)

Cadencias y Flujo Continuo

A diferencia de Scrum, Kanban no trabaja con iteraciones cerradas (Sprints) ni limpia el tablero al final de un período. El flujo es continuo e ininterrumpido.

- **Reuniones habituales**:
    - *Team Kanban Meeting*: Sincronización diaria del flujo.
    - *Team Replenishment Meeting*: Selección periódica o a demanda de nuevos ítems del backlog.
    - *Team Retrospective*: Reflexión quincenal o mensual para ajustar políticas y límites WIP.

Clases de Servicio (*Classes of Service*) y Arquetipos de Costo de Retraso

Permiten tratar de forma diferenciada la prioridad de los ítems en función de su impacto económico o riesgo de retraso:

- **Urgente / Expreso / "Bala de Plata" (Expedite)**: Emergencias críticas en producción. Tienen un límite estricto de **WIP = 1**. Suspenden el trabajo estándar para resolverse e ingresar a producción de inmediato.
- **Fecha Fija (Fixed Date)**: Ítems cuya entrega posterior a una fecha genera pérdidas directas. Riesgo asociado: Incurrir en el **"Síndrome del Estudiante"**

- **Estándar (Standard)**: Unidades de trabajo regulares sujetas a las políticas y límites de WIP habituales del tablero.
- **Intangible / Deuda Técnica**: Tareas de refactorización o mejoras de infraestructura. Si se postergan indefinidamente por falta de visibilidad para el cliente, terminarán convirtiéndose en una emergencia de tipo Urgente.

Herramienta vs. Método

Utilizar una herramienta de tableros visuales como Trello o Jira **no significa estar haciendo Kanban**. Un equipo solo aplica el Método Kanban si respeta rigurosamente los límites de WIP, opera mediante un sistema de tracción (*pull*), mide las variables de la teoría de colas y sostiene un proceso empírico de mejora continua

Métricas en los Diferentes Enfoques

Métricas Clave de Flujo en Lean / Kanban

Kanban mide la eficiencia del **proceso** a través de las variaciones de tiempo de las unidades de trabajo.

- **Lead Time (Tiempo de Entrega - Vista Externa del Cliente)**: Tiempo total que transcurre desde que el cliente solicita una característica (se deposita en la cola/backlog) hasta que la recibe terminada en producción.
- **Cycle Time (Tiempo de Ciclo - Vista Interna del Equipo)**: Tiempo transcurrido desde que el equipo asume el compromiso activo de trabajar en la tarjeta hasta que esta se completa.
- **Touch Time / Tact Time / Tiempo de Tocado**: Tiempo neto durante el cual el equipo trabaja activamente sobre la tarjeta en columnas de ejecución, excluyendo bloqueos y esperas en columnas de acumulación/buffer.
- **Eficiencia del Ciclo de Proceso (Process Cycle Efficiency)**: Muestra qué porcentaje del tiempo total del proceso aportó valor activo vs. tiempo desperdiciado en esperas.
- **Ley de Little y Diagrama de Flujo Acumulado (CFD)**: Matemáticamente, para sistemas estables, relaciona el WIP con el rendimiento (Demuestra que la única forma directa de acelerar la entrega y reducir el Lead Time es **restringir el WIP**.

Tabla Comparativa de Métricas en los 3 Enfoques de Gestión

Dimensión	**Enfoque Tradicional**	**Enfoque Ágil**	**Enfoque Lean / Kanban**
Objeto Principal de Medición	Desviación respecto al plan inicial (Costo, Tiempo, Alcance).	Capacidad de entrega de valor funcional por iteración.	Flujo continuo del proceso y rendimiento de colas.
Métricas Clave de Proceso	**Esfuerzo** (Horas/Hombre calculadas). **Tiempo / Cronograma** (Diagramas de Gantt, SPI). **Costos** (Presupuesto ejecutado, CPI).	**Velocidad** (Puntos de Historia completados por Sprint) • **Capacidad del Equipo**.	**Lead Time** (Tiempo de solicitud a entrega) **Cycle Time** (Tiempo de compromiso a fin) **Touch Time** (Tiempo activo)
Métricas de Calidad y Producto	Cantidad total de Defectos encontrados en fase de pruebas. Cumplimiento de especificaciones formales.	Running Tested Features (RTF): Funcionalidades probadas y funcionando integradamente635.	Eficiencia del Ciclo de Proceso Carga de Falla
Frecuencia de Evaluación	Hitos formales de proyecto (*Milestones*) o cierre de fases.	Cierre de cada iteración / Sprint.	Evaluación continua en tiempo real sobre el flujo.